

■ Design Patterns

Lecture Notes — Beginner Friendly

*Based on: Muhammad Zeeshan Nazar's Lecture Slides
GoF Book · Pearson 1995*

7 Patterns · UML Diagrams · Code Examples · Exam Tips ■

■ What's Inside

#	Pattern	Category	Real-Life Analogy
1	Singleton	Creational	<i>One Government per Country</i>
2	Factory Method	Creational	<i>Truck vs Ship Logistics Factory</i>
3	Adapter	Structural	<i>Voltage Converter (110V↔220V)</i>
4	Observer	Behavioral	<i>YouTube Subscriptions</i>
5	Iterator	Behavioral	<i>TV Channel Remote</i>
6	Template Method	Behavioral	<i>Data Mining: PDF vs CSV vs DOC</i>
7	State	Behavioral	<i>Document: Draft → Moderation → Published</i>

■ Chapter 1: Creational Patterns

How objects are CREATED · Singleton · Factory Method

■ Creational patterns **abstract the instantiation process** — they hide HOW objects are created, composed, and represented. The system becomes independent of how objects are made. (Lecture slides 20 & 21)

1

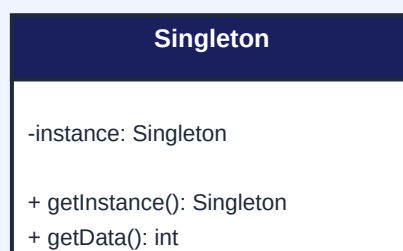
Singleton

Also known as: Virtual Instance / Single Instance

Creational

Intent	Ensures a class has only ONE instance and provides a global access point to it.
Problem	Sometimes you need to control access to a shared resource (like a database or config file). A regular constructor always returns a NEW object — you need a way to get the SAME object every time, from anywhere.
Real World	■ The Government — a country has only ONE official government. No matter who calls 'the government', they get the same institution. You can't create a second one.
Solution	1) Make the constructor private (no one else can call <i>new</i>). 2) Add a static getInstance() method. 3) First call: creates the object and stores it. Every next call: returns the same stored one.
Lazy Init	Create the instance only when first needed (getInstance() checks if instance==NULL).
Eager Init	Create the instance at program start before anyone asks. Simpler but always uses memory.
When to Use	Database connections · Logging systems · Configuration managers · Thread pools · Anything that must be shared and must be exactly ONE.

■ UML Diagram



Private constructor + static getInstance() = only one object ever

Singleton: one class, private constructor, static getInstance()

■ Key Code (C++ — from lecture)

Code Example

```
// LAZY INITIALIZATION
class Singleton {
private:
static Singleton* instance; // holds the ONE object
int data;
Singleton() { data = 0; } // private constructor!
public:
static Singleton* getInstance() {
if (instance == NULL) {
instance = new Singleton(); // created only once
}
return instance; // always same object
}
int getData() { return data; }
void setData(int d) { data = d; }
};

Singleton* Singleton::instance = NULL; // start as NULL
int main() {
Singleton* s = Singleton::getInstance();
s->setData(7);
Singleton* p = Singleton::getInstance(); // SAME object!
cout << (s == p); // Output: 1 (true – identical address)
}
```

✓ Pros

- Only one instance → saves memory
- Global access point from anywhere
- Lazy initialization → created only when needed

✗ Cons

- Hard to unit test (global state)
- Not thread-safe by default (needs mutex)
- Violates Single Responsibility Principle

2

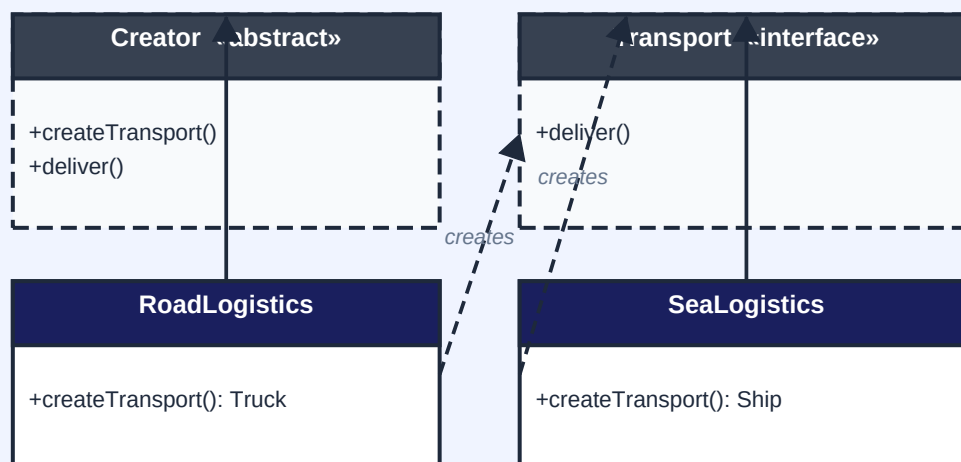
Factory Method

Also known as: Virtual Constructor

Creational

Intent	Provides an interface for creating objects in a superclass , but allows subclasses to decide which class to instantiate .
Problem	A logistics app only handles Trucks. Later, ships are needed. Most code is coupled to the Truck class. Adding Ship means changing the whole codebase — bad!
Real World	■ A Food Factory : The waiter (client) calls 'make food'. A PizzaChef makes Pizza. A PastaChef makes Pasta. The waiter doesn't know which chef or dish — just calls the interface.
Solution	Replace <code>new Truck()</code> with a factory method <code>createTransport()</code> . RoadLogistics overrides it to return a Truck. SeaLogistics overrides it to return a Ship. All products must implement the same interface (e.g. <code>deliver()</code>).
Key Rule	All products (Truck, Ship, Plane) must share a common parent/interface (e.g. Transport) so the factory can return them through one type.
When to Use	When you don't know ahead of time which class to instantiate · When subclasses should control what gets created · When adding new product types must NOT break existing code.

UML Diagram



RoadLogistics.createTransport() returns Truck | SeaLogistics returns Ship

Creator defines createTransport(). Subclasses override it to return Truck or Ship.

Key Code (C++ — from lecture)

Code Example

```
// Common interface for all products
class Vehicle {
public:
    virtual void printVehicle() = 0;
};

class TwoWheeler : public Vehicle {
public:
    void printVehicle() { cout << "I am two wheeler" << endl; }
};

class FourWheeler : public Vehicle {
public:
    void printVehicle() { cout << "I am four wheeler" << endl; }
};

// Creator – contains the factory logic
class VehicleFactory {
public:
    static Vehicle* createVehicle(int type) { // factory method
        if (type == 1) return new TwoWheeler();
        if (type == 2) return new FourWheeler();
        return NULL;
    }
};

int main() {
    Vehicle* v = VehicleFactory::createVehicle(1);
    v->printVehicle(); // Output: I am two wheeler
    Vehicle* v2 = VehicleFactory::createVehicle(2);
    v2->printVehicle(); // Output: I am four wheeler
}
```

✓ Pros

- Open/Closed Principle — add new types without changing existing code
- Reduced coupling — client doesn't know which class it gets
- Easy to extend — just add a new subclass

✗ Cons

- More classes needed (one per product type)
- Can become complex if many product types exist
- Subclassing required even for small changes

■ Chapter 2: Structural Patterns

How objects are CONNECTED · Adapter Pattern

■ Structural patterns explain how to **assemble objects and classes into larger structures** while keeping them flexible. Structural class patterns use **inheritance** to compose interfaces. (Lecture slide 22)

3

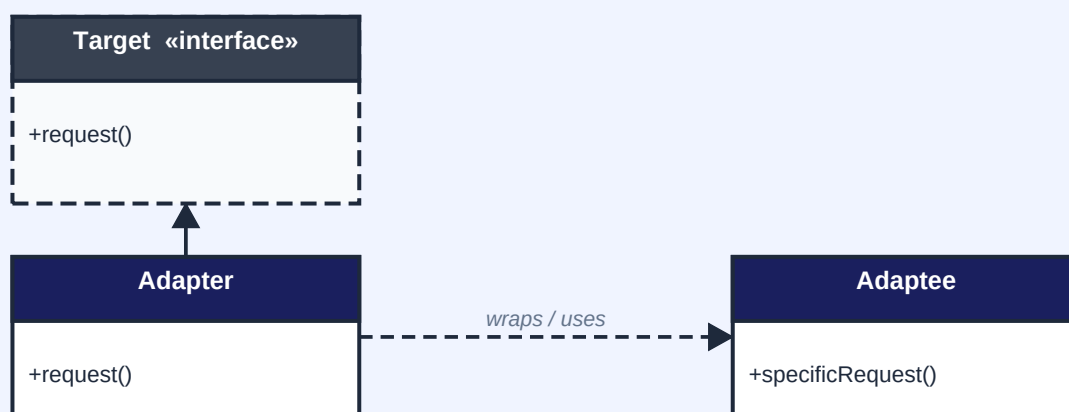
Adapter

Also known as: Wrapper

Structural

Intent	Allows objects with incompatible interfaces to collaborate by wrapping one in an adapter that translates calls.
Problem	A stock market app downloads data in XML. A new analytics library only understands JSON. You can't change the library (no source access). The interfaces are incompatible.
Real World	■ Voltage Converter : A US device runs on 110V. Pakistani sockets give 220V. An adapter converts the voltage so the device works safely — same electricity, compatible interface.
Solution	Create an XML-to-JSON Adapter . The adapter implements the interface the client expects (Target). It wraps the incompatible object (Adaptee) and translates calls. The Adaptee never knows an adapter exists.
Object Adapter	Uses composition — Adapter holds a reference to the Adaptee. Works in any language.
Class Adapter	Uses multiple inheritance — Adapter inherits from BOTH Target AND Adaptee. Only works in languages that support multiple inheritance (e.g. C++).
When to Use	When you want to use an existing class but its interface doesn't match · Integrating third-party libraries · Legacy code migration · Cross-format data conversion.

■ UML Diagrams — Object & Class Adapter



Client calls request() on Target. Adapter translates it to specificRequest() on Adaptee.

Object Adapter: Adapter implements Target, wraps Adaptee via composition

■ Code (C++ — from lecture)

Code Example

```
// Adaptee: existing class with INCOMPATIBLE interface
class Adaptee {
public:
void specificRequest() {
cout << "Adaptee's specificRequest called" << endl;
}
};

// Target: interface the client EXPECTS
class Target {
public:
virtual void request() = 0;
};

// Adapter: wraps Adaptee, implements Target
class Adapter : public Target {
private:
Adaptee* adaptee; // holds Adaptee (composition)
public:
Adapter(Adaptee* a) { adaptee = a; }
void request() override {
adaptee->specificRequest(); // translates the call
}
};

int main() {
Adaptee* adaptee = new Adaptee();
Target* adapter = new Adapter(adaptee);
adapter->request(); // Output: Adaptee's specificRequest called
}

// ---- Voltage Example (also from lecture) ----
// Device110V::operateAt110V() ← VoltageAdapter ← Device220V::operateAt220V()
// Client calls operateAt220V(), adapter converts and calls operateAt110V() internally
```

✓ Pros

- Single Responsibility — conversion logic in one place
- Open/Closed — add new adapters without touching client
- Can reuse existing incompatible classes

✗ Cons

- Code complexity increases with many adapters
- Class Adapter requires multiple inheritance (not all languages support it)
- Sometimes simpler to change the Adaptee directly

■ Chapter 3: Behavioral Patterns

How objects COMMUNICATE · Observer · Iterator · Template Method · State

■ Behavioral patterns are about **algorithms and the assignment of responsibilities between objects**. They describe not just the structure of objects or classes but also the **patterns of communication between them**. (Lecture slides 23–26)

4

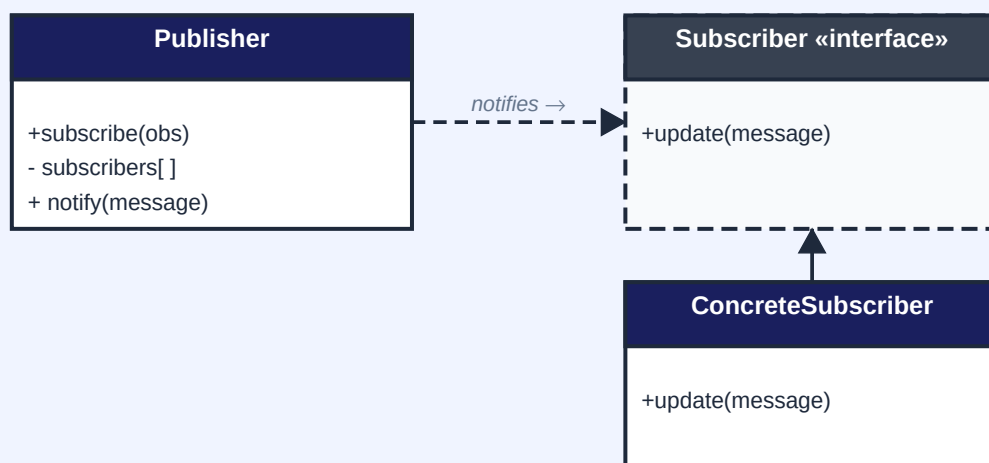
Observer

Also known as: Event-Subscriber / Listener

Behavioral

Intent	Defines a subscription mechanism to notify multiple objects about any events that happen to the object they're observing. One-to-many dependency.
Problem	A Customer wants to know when a new iPhone arrives at a Store. If the customer checks daily → waste of time. If the store emails everyone always → spam. Neither is good.
Real World	■ YouTube Subscriptions: You subscribe to a channel (Publisher). When they upload (event), YOU and all other subscribers get notified. The YouTuber doesn't track each viewer's details — just calls notify().
Solution	Publisher maintains a list of Subscribers . Subscribers implement a common interface with update() . When an event happens, publisher loops through the list and calls update() on each. Subscribers can join or leave anytime with subscribe()/unsubscribe().
Key Terms	Publisher (also: Subject, Observable) = the object being watched. Subscriber (also: Observer, Listener) = the object watching.
When to Use	When changes in one object require updating many others · Event-driven systems · GUI event listeners · Real-time notifications · MVC architecture.

■ UML Diagram



Publisher keeps a list of Subscribers and calls update() on each when state changes.

Publisher keeps subscribers[] and calls update() on each when state changes.

■ Code (C++ — from lecture)

Code Example

```
// Subscriber interface – everyone MUST implement update()
class Subscriber {
public:
virtual void update(const string& message) = 0;
};

// ConcreteSubscriber – handles the notification
class ConcreteSubscriber : public Subscriber {
public:
void update(const string& message) override {
cout << "Received update: " << message << endl;
}
};

// Publisher – maintains the list and notifies
class Publisher {
private:
list<Subscriber*> subscribers;
public:
void subscribe(Subscriber* s) { subscribers.push_back(s); }
void unsubscribe(Subscriber* s) { subscribers.remove(s); }
void notifySubscribers(const string& message) {
for (Subscriber* s : subscribers)
s->update(message); // calls every subscriber
}
};

int main() {
Publisher publisher;
ConcreteSubscriber s1, s2, s3;
publisher.subscribe(&s1);
publisher.subscribe(&s2);
publisher.subscribe(&s3);
publisher.notifySubscribers("Data updated!"); // all 3 get it
publisher.unsubscribe(&s1);
publisher.notifySubscribers("Another update!"); // only s2, s3
}
```

✓ Pros

- Open/Closed Principle — add new subscribers without changing publisher
- Establish object relationships at runtime
- Publisher doesn't need to know subscriber details

✗ Cons

- Subscribers notified in random order
- Memory leaks if subscribers don't unsubscribe
- Many notifications can cause performance issues

5

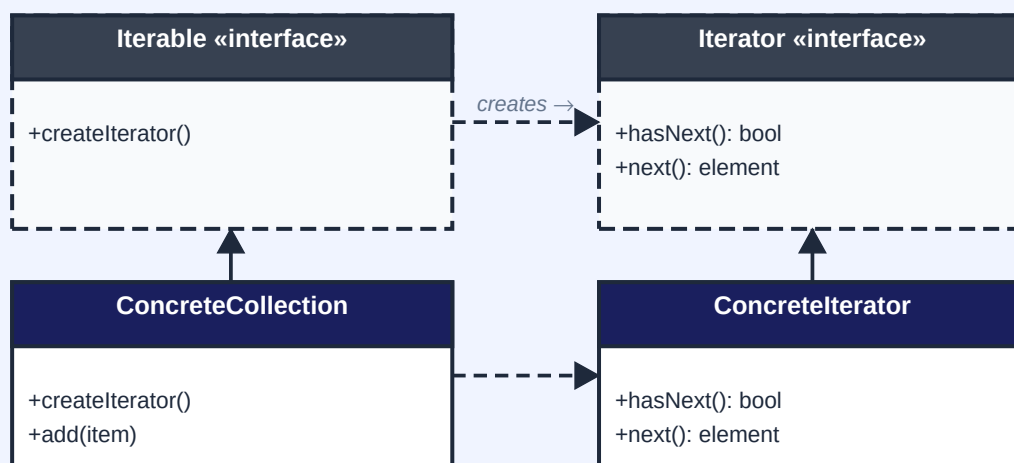
Iterator

Also known as: *Cursor*

Behavioral

Intent	Lets you traverse elements of a collection without exposing its underlying representation (list, stack, tree, graph, etc.).
Problem	Collections can be lists, trees, graphs, stacks — each stores data differently. But clients shouldn't need to know HOW data is stored. Adding traversal to the collection class bloats it with unrelated code.
Real World	■ TV Remote : You press Next Channel without knowing how the channel list is stored inside the TV (array? database? signal scan?). You just use <code>next()</code> and the TV handles the rest.
Solution	Extract traversal into a separate Iterator object . Iterator tracks current position and knows how many elements are left. Client calls <code>hasNext()</code> and <code>next()</code> — doesn't care about internals. Multiple iterators can traverse the same collection simultaneously.
Key Insight	Python's for loop , Java's enhanced for , C++'s range-for — all use the Iterator pattern internally!
When to Use	When you need to traverse without exposing internals · Multiple traversal types needed (forward, backward, random) · Uniform interface across different collection types.

■ UML Diagram



Iterator encapsulates traversal. Client calls `hasNext()/next()` without knowing the internal structure.

Two parallel hierarchies: Iterable (collection) and Iterator. Client only uses `hasNext()/next()`.

■ Code (Python — simple version)

Code Example

```
# Python's for loop already uses the Iterator pattern!
# But here's a manual example to understand it:
class NumberCollection:
    def __init__(self):
        self._numbers = []
    def add(self, num):
        self._numbers.append(num)
    def __iter__(self): # returns an iterator
        self._index = 0
        return self
    def __next__(self): # hasNext() + next() combined
        if self._index >= len(self._numbers):
            raise StopIteration # signals end
        val = self._numbers[self._index]
        self._index += 1
        return val
# Client code – doesn't know list is used internally
col = NumberCollection()
col.add(10); col.add(20); col.add(30)
for num in col: # uses __iter__ and __next__ automatically
    print(num) # Output: 10, 20, 30
# C++ version from lecture uses:
# hasNext() → checks if more elements
# next() → returns current and advances
```

✓ Pros

- Single Responsibility — traversal logic separate from collection
- Multiple iterators can traverse simultaneously and independently
- Uniform interface across all collection types

✗ Cons

- Overkill for simple collections
- Less efficient than direct index access for some structures
- More classes to manage

6

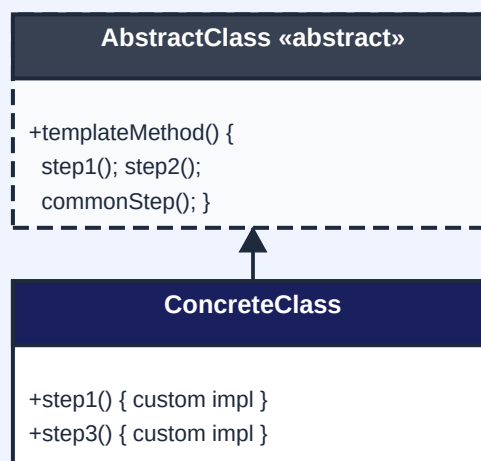
Template Method

Also known as: Template Pattern

Behavioral

Intent	Defines the skeleton of an algorithm in a superclass but lets subclasses override specific steps <i>without changing its overall structure</i> .
Problem	A data mining app processes PDF, DOC, and CSV files. All three share the same steps: open file → extract data → parse → analyze → report → close. But code for opening/extracting is different per format. Result: massive code duplication.
Real World	■ Making Tea vs Coffee : Both follow the same steps: boil water → brew → pour in cup → add extras. Only 'brew' and 'extras' differ. The template (recipe) is the same!
Solution	Create a base class with a templateMethod() that calls a fixed sequence of steps. Some steps have default implementations. Others are abstract (subclasses must implement). Some are hooks — optional empty methods subclasses can override.
Step Types	Abstract steps : MUST be implemented by subclass. Optional steps : have defaults, can be overridden. Hooks : empty by default, override for extra behavior.
When to Use	When multiple classes share the same algorithm structure · Data processors · Report generators · Game AI routines · Framework lifecycle methods.

■ UML Diagram

*AbstractClass locks the template. Subclasses only override abstract steps.**AbstractClass locks the template. Subclasses only fill in the abstract steps.*

■ Code (C++ — from lecture)

Code Example

```
// Abstract base class – defines the TEMPLATE
class AbstractClass {
public:
    // The template method – skeleton of algorithm – CANNOT override
    void templateMethod() {
        step1(); // abstract – subclass must implement
        step2(); // default implementation provided
        commonStep(); // can be overridden (optional)
        step3(); // abstract – subclass must implement
    }
    virtual void step1() = 0; // MUST implement
    virtual void step3() = 0; // MUST implement
    void step2() { // default step
        cout << "Default Step 2" << endl;
    }
    virtual void commonStep() { // hook – optional override
        cout << "Common Step (Default)" << endl;
    }
};

// Concrete class – fills in the abstract steps
class ConcreteClass : public AbstractClass {
public:
    void step1() override { cout << "Custom Step 1" << endl; }
    void step3() override { cout << "Custom Step 3" << endl; }
    void commonStep() override { cout << "Custom Common Step" << endl; }
};

int main() {
    AbstractClass* obj = new ConcreteClass();
    obj->templateMethod();
    // Output:
    // Custom Step 1
    // Default Step 2
    // Custom Common Step
    // Custom Step 3
}
```

✓ Pros

- Eliminates code duplication — shared steps in base class
- Open/Closed — add new variants by subclassing only
- Controls extension points precisely (only override what's allowed)

✗ Cons

- Clients can suppress default step by overriding it
- Template method can be hard to maintain with many steps
- Liskov Substitution Principle may be violated by some overrides

7

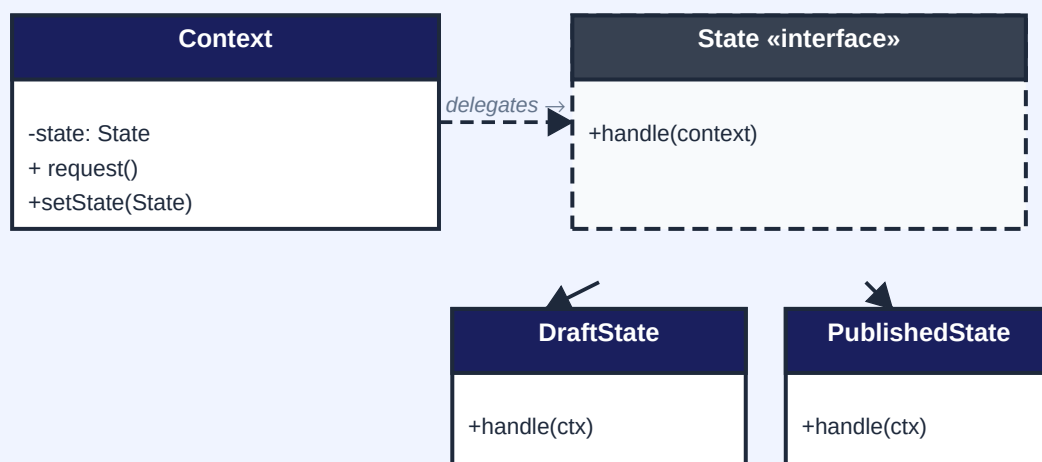
State

Also known as: *Objects for States / FSM*

Behavioral

Intent	Lets an object alter its behavior when its internal state changes . It appears as if the object changed its class.
Problem	A Document can be in 3 states: Draft, Moderation, Published. The publish() method behaves differently in each state. Implementing this with if/switch statements becomes a maintenance nightmare — any change to transition logic touches every method.
Real World	■ Traffic Light : Same physical box, completely different behavior. Red = Stop. Yellow = Slow. Green = Go. The light (context) delegates behavior to whichever state object is current.
Solution	Create a State class for each possible state (DraftState, ModerationState, PublishedState). Each State class implements state-specific behavior. The Context (Document) holds a reference to the current State and delegates work to it. To switch state: replace the State object.
vs Strategy	Strategy and State look identical in UML! The key difference is intent : State changes behavior automatically based on internal state. Strategy is swapped externally by the client.
When to Use	When behavior depends on state and must change at runtime · When you have massive if/switch based on state · Vending machines · ATMs · Order processing · Document workflows.

■ UML Diagram



Context holds current State object and delegates behavior to it. States can switch Context to another state.

Context holds current State. Each State implements handle(). States can switch the Context's state.

■ Code (C++ — adapted from lecture)

Code Example

```
// State interface
class State {
public:
    virtual void handle(class Context* ctx) = 0;
};

class Context {
private:
    State* state;
public:
    Context(State* s) { state = s; }
    void setState(State* s) { state = s; } // switch state
    void request() { state->handle(this); } // delegate to state
};

// Concrete States
class DraftState : public State {
public:
    void handle(Context* ctx) override {
        cout << "Draft: moving to Moderation..." << endl;
        ctx->setState(new ModerationState()); // transitions!
    }
};

class ModerationState : public State {
public:
    void handle(Context* ctx) override {
        cout << "Moderation: publishing if admin..." << endl;
        ctx->setState(new PublishedState());
    }
};

class PublishedState : public State {
public:
    void handle(Context* ctx) override {
        cout << "Published: document is live. No action." << endl;
    }
};

int main() {
    Context doc(new DraftState());
    doc.request(); // Draft: moving to Moderation...
    doc.request(); // Moderation: publishing...
    doc.request(); // Published: No action.
}
```

✓ Pros

- Single Responsibility — state behavior in separate classes
- Open/Closed — add new states without changing Context
- Eliminates giant if/switch conditional chains

✗ Cons

- Overkill if only a few states with simple behavior
- Many small classes created (one per state)
- State transitions spread across state classes — hard to see the full picture

■ Quick Revision Sheet

All 7 Patterns from Your Lectures — One Page!

Pattern	Category	One Sentence	Analogy
Singleton	Creational	ONE instance, global access point	<i>One government</i>
Factory Method	Creational	Subclass decides which object to create	<i>Truck vs Ship factory</i>
Adapter	Structural	Make incompatible interfaces work together	<i>110V↔220V converter</i>
Observer	Behavioral	Publisher notifies all subscribers on change	<i>YouTube subscriptions</i>
Iterator	Behavioral	Traverse collection without seeing internals	<i>TV remote Next button</i>
Template Method	Behavioral	Algorithm skeleton, subclass fills steps	<i>Tea vs Coffee recipe</i>
State	Behavioral	Object behavior changes with its state	<i>Traffic light</i>

■ Exam Tips from Your Lectures

■ Singleton vs Factory

Singleton: private constructor + static getInstance() — key giveaway in any code/diagram. **Factory Method:** superclass defines createX(), subclass overrides to pick which class.

■ Adapter: 2 Types!

Object Adapter (uses composition — Adapter HAS-A Adaptee) vs **Class Adapter** (uses inheritance — Adapter IS-A Adaptee AND IS-A Target). Class Adapter needs multiple inheritance (C++ only).

■ Observer key words

Publisher/Subject = the watched one. Subscriber/Observer/Listener = the watcher. subscribe() / unsubscribe() / notify() / update() — memorize these 4 methods.

■ State vs Strategy (same UML!)

State: object changes its own state automatically. **Strategy:** YOU (client) swap the algorithm manually. The UML looks the same — the difference is WHO changes it and WHY.

■ Template Method — the hook

Abstract steps = MUST override. Optional steps = CAN override. **Hooks** = empty methods you can optionally fill. The `templateMethod()` itself is **never overridden** — it's the locked skeleton.

■ **All the best for your exam!** These 7 patterns are directly from your teacher's slides. Remember the real-life analogies — once they click, the code makes perfect sense. You've got this! ■